

- Description

This module manages bank transfers between accounts, ensuring that:

1. The transfer amount **must be greater than zero** and **must not exceed the sender's balance**.
2. If the transfer is valid, **the amount is deducted from the sender's balance and added to the recipient's balance**.
3. If the transfer is invalid, **no changes occur**.
4. The system also **logs all successful transactions** for record-keeping.

This ensures secure and valid financial transactions between accounts.

- Specification

module BankTransferSystem

var senderBalance, recipientBalance, transferAmount: real

var transactionLog: sequence of string

process transfer(senderBalance: real, recipientBalance: real, transferAmount: real)

flag: boolean

functional scenario form:

 transferAmount <= senderBalance && flag = true and senderBalance = senderBalance - transferAmount and recipientBalance = recipientBalance + transferAmount and transactionLog = transactionLog $\oplus$ ("Transfer recorded"), Valid transfer, amount deducted and added to recipient, transaction logged

 ||

 transferAmount > senderBalance && flag = false, Invalid transfer, insufficient funds, values unchanged

 ||

 transferAmount <= 0 && flag = false, Invalid transfer, amount must be positive, values unchanged

end process

end module

- Java Code

```
package code;
import java.util.ArrayList;
import java.util.List;
class BankAccount06 {
    private String accountHolder;
    private double balance;
    public BankAccount06(String accountHolder, double initialBalance) {
        this.accountHolder = accountHolder;
        this.balance = initialBalance;
    }
    public boolean transfer(BankAccount06 recipient, double amount, TransactionLogger logger) {
```

```

        if (amount > 0 && amount <= this.balance) {
            this.balance -= amount;
            recipient.balance += amount;
            logger.logTransaction(this.accountHolder, recipient.account
Holder, amount);
            System.out.printf("Transfer successful! %s -> %s: %.2f%n",
this.accountHolder, recipient.accountHolder, amount);
            return true;
        } else {
            System.out.println("Transfer failed! Invalid amount or insu
fficient funds.");
            return false;
        }
    }
    public double getBalance() {
        return balance;
    }
    public String getAccountHolder() {
        return accountHolder;
    }
}
class TransactionLogger {
    private List<String> transactionLog = new ArrayList<>();
    public void logTransaction(String sender, String recipient, double
amount) {
        String logEntry = String.format("Transaction: %s -> %s | Amount
: %.2f", sender, recipient, amount);
        transactionLog.add(logEntry);
    }
    public void printTransactionLog() {
        System.out.println("Transaction History:");
        for (String log : transactionLog) {
            System.out.println(log);
        }
    }
}
public class c06_BankTransferSystem {
    public static void main(String[] args) {
        TransactionLogger logger = new TransactionLogger();

        // testcase-VT:
        //     BankAccount06 alice1 = new BankAccount06("Alice", 1000);
        //     BankAccount06 bob1 = new BankAccount06("Bob", 500);

```

```

//      alice1.transfer(bob1, 120.87, logger); // amount > 0 && amount
//      t <= this.balance
//      BankAccount06 alice2 = new BankAccount06("Alice", 1000);
//      BankAccount06 bob2 = new BankAccount06("Bob", 500);
//      alice2.transfer(bob2, 2640.44, logger); // amount > 0 && amount
//      > this.balance
//      BankAccount06 alice3 = new BankAccount06("Alice", 1000);
//      BankAccount06 bob3 = new BankAccount06("Bob", 500);
//      alice3.transfer(bob3, -92, logger); // amount < 0

// testcase-FT:
//      BankAccount06 alice1 = new BankAccount06("Alice", 1000);
//      BankAccount06 bob1 = new BankAccount06("Bob", 500);
//      alice1.transfer(bob1, 1.7976931348623157E308, logger); // amount
//      > 0 && amount <= this.balance
//      BankAccount06 alice2 = new BankAccount06("Alice", 1000);
//      BankAccount06 bob2 = new BankAccount06("Bob", 500);
//      alice2.transfer(bob2, 4.9E-
//      324, logger); // amount > 0 && amount > this.balance
//      BankAccount06 alice3 = new BankAccount06("Alice", 1000);
//      BankAccount06 bob3 = new BankAccount06("Bob", 500);
//      alice3.transfer(bob3, 3.313394118237622E307, logger); // amount
//      < 0

// testcase-Z3:
//      BankAccount06 alice1 = new BankAccount06("Alice", 1000);
//      BankAccount06 bob1 = new BankAccount06("Bob", 500);
//      alice1.transfer(bob1, 16, logger); // amount > 0 && amount <=
//      this.balance
//      BankAccount06 alice2 = new BankAccount06("Alice", 1000);
//      BankAccount06 bob2 = new BankAccount06("Bob", 500);
//      alice2.transfer(bob2, -
//      1, logger); // amount > 0 && amount > this.balance
//      BankAccount06 alice3 = new BankAccount06("Alice", 1000);
//      BankAccount06 bob3 = new BankAccount06("Bob", 500);
//      alice3.transfer(bob3, 1, logger); // amount < 0

// testcase-UVT:
BankAccount06 alice1 = new BankAccount06("Alice", 1000);
BankAccount06 bob1 = new BankAccount06("Bob", 500);
alice1.transfer(bob1, 120.87, logger); // amount > 0 && amount
<= this.balance
BankAccount06 alice2 = new BankAccount06("Alice", 1000);
BankAccount06 bob2 = new BankAccount06("Bob", 500);

```

```

        alice2.transfer(bob2, 44, logger); // amount > 0 && amount > th
is.balance
        BankAccount06 alice3 = new BankAccount06("Alice", 1000);
        BankAccount06 bob3 = new BankAccount06("Bob", 500);
        alice3.transfer(bob3, -92, logger); // amount < 0

        // print log
        logger.printTransactionLog();
    }
}

```

- Coverage report

1. Test case number: 3
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.7 %	6	810	816
✓ src	0.7 %	6	810	816
✓ code	0.7 %	6	810	816
✓ c06_BankTransferSystem.java	100.0 %	6	0	6
> BankAccount06	100.0 %	4	0	4
> c06_BankTransferSystem		0	0	0
> TransactionLogger	100.0 %	2	0	2

3. Vibration testing instruction coverage: 95.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	1.1 %	177	15,333	15,510
✓ src	1.1 %	177	15,333	15,510
✓ code	1.1 %	177	15,333	15,510
✓ c06_BankTransferSystem.java	95.2 %	177	9	186
> BankAccount06	91.5 %	65	6	71
> c06_BankTransferSystem	95.3 %	61	3	64
> TransactionLogger	100.0 %	51	0	51

4. Fuzz testing branch coverage: 75%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.6 %	5	811	816
✓ src	0.6 %	5	811	816
✓ code	0.6 %	5	811	816
✓ c06_BankTransferSystem.java	83.3 %	5	1	6
> BankAccount06	75.0 %	3	1	4
> c06_BankTransferSystem		0	0	0
> TransactionLogger	100.0 %	2	0	2

5. Fuzz testing instruction coverage: 95.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	1.1 %	177	15,333	15,510
✓ src	1.1 %	177	15,333	15,510
✓ code	1.1 %	177	15,333	15,510
✓ c06_BankTransferSystem.java	95.2 %	177	9	186
> BankAccount06	91.5 %	65	6	71
> c06_BankTransferSystem	95.3 %	61	3	64
> TransactionLogger	100.0 %	51	0	51

6. Z3 branch coverage: 83.3%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.6 %	5	835	840
▼ src	0.6 %	5	835	840
▼ code	0.6 %	5	835	840
▼ c06_BankTransferSystem.java	83.3 %	5	1	6
> BankAccount06	75.0 %	3	1	4
> TransactionLogger	100.0 %	2	0	2
> c06_BankTransferSystem		0	0	0

7. Z3 instruction coverage: 95.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.8 %	177	21,909	22,086
▼ src	0.8 %	177	21,909	22,086
▼ code	0.8 %	177	21,909	22,086
▼ c06_BankTransferSystem.java	95.2 %	177	9	186
> BankAccount06	91.5 %	65	6	71
> c06_BankTransferSystem	95.3 %	61	3	64
> TransactionLogger	100.0 %	51	0	51

8. Unimproved V-method branch coverage: 83.3%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.6 %	5	835	840
▼ src	0.6 %	5	835	840
▼ code	0.6 %	5	835	840
▼ c06_BankTransferSystem.java	83.3 %	5	1	6
> BankAccount06	75.0 %	3	1	4
> TransactionLogger	100.0 %	2	0	2
> c06_BankTransferSystem		0	0	0

9. Unimproved V-method instruction coverage: 95.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.8 %	177	21,909	22,086
▼ src	0.8 %	177	21,909	22,086
▼ code	0.8 %	177	21,909	22,086
▼ c06_BankTransferSystem.java	95.2 %	177	9	186
> BankAccount06	91.5 %	65	6	71
> c06_BankTransferSystem	95.3 %	61	3	64
> TransactionLogger	100.0 %	51	0	51